CodeProctor

A Web-Based Coding Assessment and Malpractice Detection Platform

Modules and User Stories

Team No: 15

Team Members:
Abhinand M A (04)
Mathew Punnen Chandy (40)
Sreelakshmi K S (56)

Modules and User Stories

The platform is broken down into the following functional modules. Each module lists its user stories in the standard "As a [role], I want to [action], so that [benefit]" format, reflecting the three core roles: Admin, Contest Creator, and Participant.

Module 1 — Authentication & Role-Based Access Control (RBAC)

JWT-based authentication with roles resolved contextually per contest (Admin / Contest Creator / Participant) rather than a fixed global role.

- As a new user, I want to register with my name, email, and password, so that I can create an account on the platform.
- As a registered user, I want to log in securely, so that I can access my dashboard and contests.
- As a logged-in user, I want the system to determine my role contextually for each contest (owner, participant, or neither), so that I only see actions I'm permitted to perform.
- As a user, I want my session to be authenticated via JWT on every API request, so that my account and submissions remain secure.
- As a platform administrator, I want a separate admin-level permission, so that I can manage public problems and view platform-wide submissions.

Module 2 — Problem Bank

CRUD management of coding problems, test cases, constraints, and difficulty tags — covering both public practice problems and contest-specific questions.

- As an Admin, I want to create public problems with a description, difficulty tag, and test cases, so that all users can practice outside of contests.
- As an Admin, I want to edit or remove public problems, so that I can keep the practice bank accurate and up to date.
- As a Participant, I want to browse and filter public problems by difficulty or tag, so that I can find problems suited to my skill level.
- As a Participant, I want to solve public problems at any time without a timer or proctoring, so that I can practice freely.
- As a Contest Creator, I want to author new questions with input/output test cases, time limits, and memory limits, so that I can build a contest tailored to my requirements.
- As a Contest Creator, I want to mark certain test cases as hidden, so that participants cannot see the exact grading inputs.

Module 3 — Contest Engine

Contest creation, scheduling, question assignment, and participant registration — open to any registered user, not just a central admin.

- As a Contest Creator, I want to create a contest with a title, start time, and end time, so that I can host a scheduled assessment.
- As a Contest Creator, I want to choose a proctoring mode (Normal, Full-Screen Enforced, or CV-Proctored) when creating a contest, so that I can set the appropriate level of monitoring.
- As a Contest Creator, I want to add existing or newly authored questions to my contest, so that participants are evaluated on the right problems.

- As a Contest Creator, I want to configure who can view or join my contest, so that I can control access.
- As a Participant, I want to register for contests, so that I can take part in assessments created by other users.
- As a Participant, I want to see the proctoring requirements of a contest before I join, so that I know what will be monitored.

Module 4 — Judge (Code Execution)

Sandboxed, resource-limited execution of submitted code using an ephemeral Docker container per submission, coordinated through a Redis-backed worker queue.

- As a Participant, I want to submit my code in a supported language, so that it is evaluated against the problem's test cases.
- As a Participant, I want my code to run in an isolated, resource-limited environment, so that faulty or malicious code cannot affect the host system or other users.
- As a Participant, I want to see a live status update (queued, running, verdict) for my submission, so that I know the outcome without refreshing the page.
- As a Participant, I want to receive a clear verdict (Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error, etc.) for each submission, so that I understand why I passed or failed.
- As a system, submissions are pushed to a Redis queue and processed by a pool of worker processes running Docker containers with CPU, memory, timeout, and no-network restrictions, so that execution stays safe and predictable under load.

Module 5 — Proctoring: Full-Screen & Browser Signals

Client-side detection of full-screen exits, tab switches, and window focus loss, logged as timestamped timeline events whenever Full-Screen or CV mode is enabled.

- As a Contest Creator, I want to enforce full-screen mode for my contest, so that participants cannot easily switch to other applications unnoticed.
- As a Participant, I want to be clearly notified that full-screen enforcement is active before I start, so that I understand what is being monitored.
- As the system, I want to log an event whenever a participant exits full-screen or switches tabs/loses window focus, so that the creator has a factual record rather than a black-box flag.
- As a Contest Creator, I want an optional copy-paste-into-editor detection signal, so that I can flag potential external code pasting.

Module 6 — Proctoring: Computer Vision (CV)

A separate Python/FastAPI/OpenCV microservice that analyzes low-frequency webcam frame snapshots for face presence, multiple faces, and gaze diversion, only when CV mode is enabled by the contest creator.

- As a Contest Creator, I want to enable CV-based proctoring for a contest, so that webcam monitoring is required for participants.
- As a Participant, I want to be asked for explicit camera and microphone permission before entering a CV-proctored contest, so that I can give informed consent.
- As the system, I want to capture webcam frames at a low frequency (e.g., every 2–3 seconds) and send them to the CV service, so that monitoring is bandwidth-light and simple.
- As the CV service, I want to detect and log 'no face detected for more than N seconds', so that prolonged absence is flagged.

- As the CV service, I want to detect and log 'multiple faces detected in frame', so that potential unauthorized assistance is flagged.
- As the CV service, I want to detect and log 'gaze/face turned away from the screen for more than N seconds', so that sustained distraction is captured.

Module 7 — Leaderboard

Real-time, per-contest ranking based on score and time-based tiebreakers, powered by Redis sorted sets and pushed live via Socket.IO.

- As a Participant, I want to see a live leaderboard for the contest I've joined, so that I know my current standing.
- As a Participant, I want the leaderboard to update in real time as I and others submit solutions, so that I don't need to refresh the page.
- As a Contest Creator, I want ties to be broken using submission time, so that ranking is fair and deterministic.
- As a Participant, I want to see my own rank and score clearly highlighted, so that I can track my progress at a glance.

Module 8 — Malpractice Timeline

A per-user, per-contest chronological log of every flagged event (full-screen and CV based), giving contest creators explainable, evidence-based review instead of a single cheating flag.

- As a Contest Creator, I want to view a timestamped timeline of all flagged events for each participant, so that I can review potential malpractice with full context.
- As a Contest Creator, I want each event to show its type, severity, and timestamp (and a snapshot reference where available), so that I can judge the seriousness of the incident myself.
- As a Contest Creator, I want to disqualify a participant based on timeline evidence, so that I can enforce fair play with a documented reason.
- As a Participant, I want disqualification decisions to be based on visible, explainable events rather than a hidden algorithm, so that the process feels transparent.

Module 9 — Admin Dashboard

Platform-wide oversight for administrators, separate from any individual contest creator's dashboard.

- As an Admin, I want a dashboard listing all public problems, so that I can manage the practice problem bank.
- As an Admin, I want to view all submissions across the platform, so that I can monitor overall system usage and integrity.
- As an Admin, I want to manage user accounts at a platform level, so that I can address abuse or support requests.